Reproducing and Critiquing LGEA and MAKAR

Two Methods that Use Large Language Models on Multimodal Graph Data (EMNLP 2025)

Châu Ngọc Huy* Hoàng Minh Thắng†
Faculty of Information Technology, University of Science, VNU-HCM
Course: Machine Learning with Graph Data (HP3-K35)
*Student ID 25C11008 †Student ID 25C11020

***Abstract*—This report studies, reproduces and critiques two EMNLP 2025 papers that use large language models (LLMs) on graph and multimodal data: LGEA for multi-modal entity alignment and MAKAR for grounded multimodal named entity recognition. For each paper we restate the task and method in our own words, rerun the main result tables on two Kaggle T4 GPUs (the authors used an A100 or 8 A800 GPUs), and check every number against the paper. For LGEA, we reproduce Hits@1 to within 1–2.5 points of the paper on the three DBP15K splits, and all seven ablation settings on FBDB15K with the same ranking of component importance, even though we had to rebuild the whole data pipeline and could not run the image-filtering module. For MAKAR, we reproduce both datasets and match the paper's 3B grounding results to within about one EEG point. On FMNERG we obtain an MNERG 3.2 points above the paper and show that the auxiliary knowledge the authors released for its test set shows signs of gold-label leakage; on GMNER our ablation rows match the paper's while our full system does not, which suggests the paper's ablation gaps overstate the contribution of its correction and knowledge agents. For each main claim of both papers, we lay out what the evidence shows, what it does not isolate, and which experiment would settle it.**

***Index Terms*—multi-modal entity alignment, multi-modal knowledge graph, grounded named entity recognition, large language models, reproducibility**

I. Introduction

Background. A multi-modal knowledge graph (MMKG) attaches images and text to the entities of a conventional knowledge graph. Two basic tasks for building and connecting such graphs are aligning entities across two graphs, and extracting entities from raw image–text data to put into a graph. In 2025 both tasks shared a trend: an LLM is used not as the final classifier but as a *data cleaning and enrichment* step before the graph model or the sequence tagger does its work.

The two papers. The first, *"Breaking the Noise Barrier: LLM-Guided Semantic Filtering and Enhancement for Multi-Modal Entity Alignment"* (LGEA) [1], uses an LLM to drop noisy images and summarize attributes before aligning two MMKGs. The second, *"MAKAR: a Multi-Agent framework based Knowledge-Augmented Reasoning for Grounded Multi-modal Named Entity Recognition"* [2], organizes three agents built on multimodal LLMs to recognize entities in tweets and locate them in the attached image. Both papers come from the same venue, both use an LLM as a source of knowledge for preprocessing, and both raise the question we care about: where does the improvement come from, and what is the result sensitive to?

Goals. For each paper we set three goals: (1) understand the task, idea and architecture and restate them in our own words; (2) rerun the main result tables on the resources we have, two 15 GB Kaggle T4 GPUs, and compare every number with the paper; (3) from that process, identify what the paper leaves unsaid and which claims its evidence does not fully support.

Scope and constraints. Compute is the deciding factor. The authors ran LGEA on an A100 40 GB and MAKAR on 8×A800. On a T4 we have no bf16, sessions are capped at 12 hours and 30 hours per week. Some components therefore could not be rerun; each experiment section says which. Reproduction code, notebooks and rerun instructions are at https://github.com/MinhThang03/graphML [3]; training logs were imported into Weights & Biases from the notebook outputs and are linked in each results section, with the two project addresses in Appendix C.

Organization. Sections II–V cover LGEA: task, method, reproduction and critique. Sections VI–IX repeat this structure for MAKAR. Section X summarizes what the two papers have in common.

II. LGEA: the multi-modal entity alignment task

A. Problem statement

An MMKG is a tuple $\mathcal{G} = (\mathcal{E}, \mathcal{R}, \mathcal{A}, \mathcal{V}, \mathcal{I}, \mathcal{T})$ of entities, relations, attributes, attribute values, images and triples. There are two kinds of triples: relation triples (e_h, r, e_t) connect two entities, and attribute triples (e, a, v) attach a value (number, date, string) to an entity. Given two graphs $\mathcal{G}_1$, $\mathcal{G}_2$ and a small set of seed pairs known to be equivalent, multi-modal entity alignment (MMEA) asks for every remaining pair (e_1, e_2) that refers to the same real-world object.

In practice each entity is encoded as a vector and alignment becomes retrieval: for e_1, rank every e_2 by cosine similarity. The metrics are Hits@k (the fraction of gold pairs whose correct e_2 is in the top k) and MRR. The test set is the gold pairs outside the seed, so the model must generalize from the 20–30 % of known pairs to the remaining 70–80 %.

B. Data and available signals

Two standard dataset families are used. DBP15K [4] contains three bilingual graph pairs (ZH-EN, JA-EN, FR-EN),

each with 15,000 gold pairs and 30 % used as seed. FBDB15K and FBYG15K [5] link Freebase to DBpedia or YAGO, with 12,846 and 11,199 gold pairs respectively and 20 % used as seed. Each entity has up to four signals: neighbourhood structure, its set of relations, its set of attributes, and an image (as a precomputed ResNet feature). The last two are where noise appears: an image may be unrelated to the entity, and attributes may repeat or be bare numbers and dates that do not distinguish entities.

C. The gap the paper targets

Methods before LGEA, such as MCLEA [6] and MEAformer [7], focus on *how to fuse* the four signals: per-modality contrastive learning followed by attention weights. They assume the inputs are clean. Masked-MMEA [8] was the first to filter images, but it relies on ontology-based image classification and a hand-built class-conflict dictionary, which is coarse and does not generalize. TMEA [9] feeds attribute–value pairs directly to an LLM for matching, which according to the LGEA authors leads to false matches because values repeat. The gap LGEA targets is *reducing noise at the semantic level before fusion*, by letting an LLM decide which images to keep and which attributes are worth stating.

III. LGEA: method

Fig. 1 summarizes the data flow. We split LGEA into two layers: an *LLM preprocessing* layer, which is the paper's contribution, and an *alignment learning* layer inherited almost unchanged from MEAformer [7]. This split is what makes the ablation table readable later: the three rows *w/o llm*, *w/o mask* and *w/o it* measure the first layer and the training scheme, while the other four measure the value of each modality in the second layer.

A. Semantic image filtering with an LLM

Instead of asking "which class does this image belong to" as Masked-MMEA does, LGEA asks "are the image of e_1 and the image of e_2 about the same topic". To let a text-only LLM answer a question about images, the authors go through captions: BLIP [10] generates three captions per image, and GPT-3.5 receives two captions and answers Yes/No (prompt in Table 12 of the paper). The answer becomes a binary mask m_e with the same dimension as the image vector:

$$\tilde{v}_e = m_e \odot v_e, \qquad h_e^V = \mathrm{FC}(\tilde{v}_e), \tag{1}$$

where v_e is the precomputed ResNet feature and m_e is all ones if the LLM says Yes and all zeros if No. The mask therefore has only two states: keep the image as is, or remove it entirely. There is no soft weighting. What we find notable is that the question is asked about *a pair* of entities, so the filter needs to know which e_2 goes with e_1; we return to this in Section V.

B. Attribute summarization with an LLM

Instead of feeding the list $(a_1, v_1), (a_2, v_2), \ldots$ directly into the encoder, LGEA does two things. First, LLaMA-3-8B-Instruct [11] reads that list, with the entity name removed, and writes an English description of at most 100 words, denoted t_e (prompt in Table 11 of the paper; summarized in our Appendix). The 100-word limit forces the model to keep salient attributes and drop repeated ones. Second, the same LLaMA-3 serves as the text encoder through LLM2Vec [12], producing a 4096-dimensional vector:

$$h_e^A = \mathrm{FC}\big(F_{\mathrm{LLM}}(t_e)\big). \tag{2}$$

This is where LGEA uses more parameters than earlier methods (Fig. 2 of the paper): the FC layer projects from 4096 down to 300 dimensions.

C. Modality encoding and fusion

The other three modalities use familiar encoders: a one-layer GAT [13] for structure, a 1000-dimensional bag-of-relations vector through an FC layer for relations, and an FC layer for the filtered image. The four vectors h_m are stacked and passed through multi-head cross-modal attention (MHCA). The attention weights are aggregated over heads and softmax-normalized over modalities to give α_m, and

$$h_{\text{joint}} = \bigoplus_{m \in M} \alpha_m \cdot \mathrm{Norm}(h_m), \tag{3}$$

where $\bigoplus$ is concatenation and Norm is ℓ_2 normalization. So α_m does not select modalities; it rescales each block before concatenation. A modality judged less reliable is still present in h_{joint}, only smaller.

D. The three loss terms

For a seed pair (e_1, e_2) and an embedding function f, the in-batch contrastive probability is

$$p(e_1, e_2) = \frac{\exp(\mathrm{sim}(f(e_1), f(e_2))/\tau)}{\sum_{e' \in N(e_1, e_2)} \exp(\mathrm{sim}(f(e_1), f(e'))/\tau)}, \tag{4}$$

and $\mathcal{L}_m = -\mathbb{E}[\log p_m(e_1, e_2) + \log p_m(e_2, e_1)]$ is the bidirectional loss. The three terms of $\mathcal{L}$ differ in what f is and what it is compared against:

- $\mathcal{L}_{\text{intra}} = \sum_m \lambda_m \mathcal{L}_m$: $f = h_m$, each modality pulls gold pairs together on its own; λ_m is learned.
- $\mathcal{L}_{\text{joint}}$: $f = h_{\text{joint}}$, the same formula on the fused vector.
- $\mathcal{L}_{\text{IAL}} = \lambda \sum_m \frac{1}{2}[\mathrm{KL}(q_o \| q_m) + \mathrm{KL}(q_o(e_2, e_1) \| q_m(e_2, e_1))]$: q_o is the alignment distribution of the fused vector and q_m that of modality m; this term pulls each modality toward the same "opinion" as the fused vector.

According to Table 5 of the paper, removing $\mathcal{L}_{\text{intra}}$ hurts the most (0.801 down to 0.711) and removing $\mathcal{L}_{\text{IAL}}$ the least (down to 0.770).

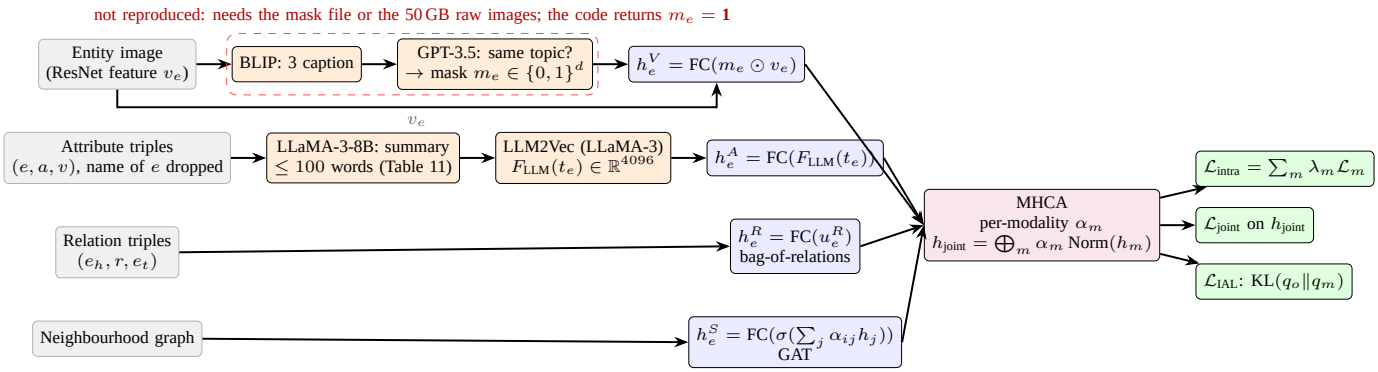


Figure 1: Data flow in LGEA as we read Fig. 1 and Section 4 of the original paper. Both LLM steps sit entirely before learning: they change the *inputs* of the image and attribute encoders, and the rest is inherited from MEAformer.

E. Iterative learning

The paper mentions it in a single sentence of the setup: "the models are trained iteratively, gradually incorporating the predictions from previous rounds to refine the embedding space". In the code [14] this is the flag `--il --il_start 500`: after epoch 500, every 5 epochs the model takes the pairs on which retrieval agrees in both directions, adds them to the seed set, and keeps training until epoch 1000. We describe it separately because in Section IV it turns out to be the fourth-largest contributor, worth more than images and relations combined.

IV. LGEA: reproduction

A. What the original repository does not provide

The repository [14] contains only the model. Three things needed to run it were not released: the per-graph attribute files (`attribute1.txt`, `attribute2.txt`, `ent2id`), the image-mask file produced by BLIP and GPT-3.5, and the intermediate caption file. We rebuilt the pipeline from three independent sources: graph structure and ResNet image features from the MEAformer data package; DBP15K attribute values from the raw N-Triples dumps; and FB15K, DB15K and YAGO15K attribute values from the `mmkb` repository. Our two scripts, `build_attributes.py` and `build_attributes_fb.py`, stop with an error when entity indices disagree between the attribute source and the image-feature file, because a run with misaligned indices silently produces meaningless numbers.

The image mask could not be rebuilt. BLIP needs pixels, while the data package only contains compressed ResNet vectors; the roughly 50 GB of raw images are only distributed through Baidu Cloud. When no mask path is given, the mask loader in `model/NewEA.py` returns an all-ones vector, so every image is kept. This is exactly the definition of the *w/o mask* row in Table 3 and the 0%mask row in Table 4 of the paper. The image branch still works; only the per-image on/off switch is missing. All our LGEA results are therefore for the *w/o mask* setting and are compared with the corresponding row of the paper.

B. Setup and forced deviations

Table I lists the setup. Hyperparameters follow the commands in the authors' README: 500 epochs for DBP15K, 1000 epochs with iterative learning for FBDB15K, lr 5×10^{-4}, batch 3500, $\tau = 0.1$, CSLS $k = 3$ [15], seed 42, one run per setting. The ten-month-old code needed six patches to run on current libraries and on two 15 GB cards; details are in Appendix A. None of them changes the model: five concern library compatibility and GPU placement, and the sixth rewrites the gradient of the sparse matrix product in the GAT from $O(N^2)$ memory to $O(E \cdot d)$, checked to give identical values before use.

The three notebooks that produce the tables below are in the LGEA/notebooks/ folder of our repository [3]: two DBP15K notebooks (reproduce-lgea-emnlp-2025-dbp15k-zh-en.ipynb and reproduce-lgea-emnlp-2025-dbp15k-fr-en.ipynb) and the FBDB15K notebook (reproduce-lgea-table-2-table-3-fbdb15k-fbyg.ipynb).

C. Table 1: DBP15K

Table II compares the three splits. We are 1.0 to 2.5 points below the paper on Hits@1, while Hits@10 differs by at most 0.33 points. With a single run, no image mask, and a rebuilt attribute file that covers only 77 % of entities, we consider this a successful reproduction. Training converges within about the first 50 epochs and then plateaus; per-epoch curves of the three runs on W&B: ZH-EN, JA-EN, FR-EN.

Before training, we ran a test that is not in the paper: alignment using only the attribute vector $F_{\mathrm{LLM}}(t_e)$ with cosine similarity, with no learning, no structure and no image (Table III). This signal alone gives Hits@1 0.872 on ZH-EN, the same as LoginMEA [16], the strongest pre-LGEA method in the paper's own Table 1. We discuss what this number means in Section V.

Table I: LGEA reproduction setup and differences from the paper.

	Paper	Ours
Hardware	A100 40 GB	Kaggle T4×2, 15 GB/card, fp16
Summarizer, encoder	LLaMA-3-8B-Instruct, LLM2Vec	same, layers split over 2 cards
Image filter	BLIP + GPT-3.5	none; $m_e = \mathbf{1}$ (= *w/o mask*)
Attributes	preprocessed by the authors, not released	rebuilt from raw data, covers ≈ 77 % of DBP15K entities
Entity names	not used	`use_surface 0`, names removed from triple identifiers
Runs	not stated	1 run, seed 42
Time	5–8 h LLM (Table 8)	3.5 h LLM + 0.5 h training per DBP15K split; 11.3 h for the 7 FBDB15K settings

Table II: DBP15K (30 % seed): numbers reported in Table 1 of the original paper versus our reproduction (l2r direction, no image mask). Δ in percentage points.

	Hits@1			Hits@10		MRR	
Split	Paper	Ours	Δ	Paper	Ours	Paper	Ours
ZH-EN	0.970	0.9578	-1.2	0.998	0.9974	0.982	0.975
JA-EN	0.962	0.9518	-1.0	0.999	0.9957	0.977	0.969
FR-EN	0.975	0.9501	-2.5	0.999	0.9963	0.985	0.968

D. Tables 2 and 3: FBDB15K and seven ablation settings

Table IV is our main LGEA result. Our LGEA row is 1.8 points Hits@1 above the paper's *w/o mask* row. Our six other ablation rows are all below the paper, but for those rows the paper *still uses* the image mask and we do not, so the absolute gaps are not comparable; what we can compare is the *ranking* of the drops. Table V shows the ranking matches in 6 of 6 positions: structure > attributes > LLM > iterative learning > relations > image. Per-epoch logs of all seven runs on W&B: LGEA, *w/o img*, *w/o rel*, *w/o structure*, *w/o att*, *w/o llm*, *w/o it*.

The last column of Table IV gives a cross-check we find reliable: Hits@1 of the LGEA row at epoch 500, just before iterative learning starts, is 0.5919; the independent *w/o it* run, which disables iterative learning and stops at 500 epochs, gives 0.5987. The two measurements differ by 0.7 points and agree. Since each setting was run only once with seed 42, we treat differences below about 2 points in this section as within an unmeasured noise band.

E. What was not run

FBYG15K was not run because the GPU quota ran out; we chose to run all seven rows on one dataset rather than one row on two datasets. The masked rows (full LGEA, 50%mask, 80%mask) could not be run for the reason above. Table 5 (loss ablation), Table 6 (LLM swap) and Table 7 (learning rate) are out of scope.

Table III: Alignment with LLM2Vec attribute vectors only (cosine, no training, no structure, no image) over all gold pairs of DBP15K.

Split	Hits@1	Hits@10	MRR	Pairs
ZH-EN	0.872	0.966	0.907	14 999
JA-EN	0.845	0.939	0.880	14 998
FR-EN	0.853	0.952	0.890	14 997

V. LGEA: analysis and critique

How to read this section. We do not judge LGEA by whether its final numbers are high or low, but by one question: *which experiment supports each claim of the paper, and does that experiment actually isolate the contribution of the module being credited?* Table VI summarizes the five main claims. Each subsection then follows the same steps: what the paper says, the evidence the authors give, why that evidence is not enough, and the experiment needed to settle it.

Table VI: The five main claims of LGEA: the authors' evidence, what that evidence does not establish, and our verdict.

Claim	Evidence → gap	Verdict
K1. LLM attribute summaries add semantics	*w/o llm* drops 11.9 points → that row changes both the summarizer and the encoder; names are present in the attributes	Not established
K2. BLIP + GPT removes the right noisy images, better than CLIP	*w/o mask* drops 8.7 points → filter accuracy never measured, no CLIP comparison, captions uncontrolled	Not established
K3. The filtering ratio is near-optimal; only about 10 % of images are needed	Table 4 → the actual ratio is not reported; the "x%mask" definition is self-contradictory	Not verifiable
K4. Entity names are not used	Names removed from identifiers → names remain as DBP15K attribute values	Does not hold on DBP15K
K5. SOTA comes from the two LLM modules	Tables 1, 2 → iterative learning adds 13 points but is not described; two different number sets	Partly true

A. K1. Is summarizing attributes really better than raw attributes?

The paper says. Raw attributes are noisy and redundant; letting an LLM summarize them into a paragraph of at most 100 words and then encoding it gives a semantically richer representation.

Evidence. The *w/o llm* row in Table 3 of the paper drops 11.9 points Hits@1 on FBDB15K and 29.2 points on FBYG15K. In Table 6, replacing LLaMA-3 with LLaMA-2 or Mistral barely changes the result (0.791–0.801).

Why it is not enough. Compared with LGEA, the *w/o llm* row differs in *two* ways at once: there is no summarization

Table IV: FBDB15K (20 % seed, 1000 epochs, iterative learning from epoch 500): Tables 2 and 3 of the original paper versus ours. The paper's ablation rows keep the image mask; ours do not. The last column is Hits@1 at epoch 500, just before iterative learning starts.

	Hits@1			Hits@10		MRR		Hits@1
Setting	Paper	Ours	Δ	Paper	Ours	Paper	Ours	epoch 500
LGEA full (with mask)	0.801	–	–	0.910	–	0.842	–	–
LGEA = *w/o mask*	0.714	0.7322	+1.8	0.868	0.8798	0.772	0.786	0.5919
w/o img	0.740	0.7147	-2.5	0.890	0.8640	0.791	0.767	0.6277
w/o rel	0.737	0.6688	-6.8	0.890	0.8455	0.791	0.732	0.5203
w/o structure	0.440	0.4030	-3.7	0.615	0.5764	0.502	0.466	0.3478
w/o att	0.655	0.5402	-11.5	0.840	0.7646	0.720	0.621	0.3688
w/o llm	0.682	0.5570	-12.5	0.855	0.7834	0.745	0.637	0.3797
w/o it	0.684	0.5987	-8.5	0.868	0.8192	0.749	0.676	–

Table V: Drop in Hits@1 (percentage points) when removing each component, each side against its own full setting: the paper against 0.801, ours against 0.7322. Ordered as in the paper.

#	Removed	Paper	Ours
1	*w/o structure*	-36.1	-32.9
2	*w/o att*	-14.6	-19.2
3	*w/o llm*	-11.9	-17.5
4	*w/o it*	-11.7	-13.3
5	*w/o rel*	-6.4	-6.3
6	*w/o img*	-6.1	-1.7

step, and the 4096-dimensional LLM2Vec encoder is not used. The paper does not say what *w/o llm* uses instead; we took it to be MEAformer's 1000-dimensional bag-of-attributes encoding and ran it that way. The large drop may therefore come from the stronger encoder rather than from summarization. Table 6 does not separate the two factors either, since each LLM swap changes both the summarizer and the encoder. Moreover, our test (Table III) shows that the attribute vector alone reaches Hits@1 0.872 on DBP15K ZH-EN without any learning, because entity names sit in the attribute values (Section V-D). Part of the "richer semantics" may simply be the LLM copying the name.

What we could verify. Our reproduction points the same way as the paper: removing the LLM costs 17.5 points Hits@1 on FBDB15K. This confirms that the *combination* "summary + LLM2Vec" beats bag-of-attributes, but not the role of the summarization step on its own.

Experiments needed. (a) Feed the raw attribute list directly into LLM2Vec without summarizing: if the result is close to LGEA, the credit belongs to the encoder. (b) Summarize with the LLM but encode with a small sentence encoder. (c) Repeat on DBP15K after removing the `name` predicate, to rule out the effect of names.

B. K2. Does the BLIP + GPT filter really remove the noisy images?

The paper says. For each aligned entity pair, BLIP captions the two images and GPT-3.5 answers whether the two captions are on "the same topic"; if not, the image is removed (all-zero mask). This is finer than ontology-based classification and needs no threshold, unlike CLIP.

Evidence. Removing the mask lowers FBDB15K Hits@1 from 0.801 to 0.714. Table 4 compares three mask levels: 0%, 50% and 80%.

Why it is not enough. Both pieces of evidence are *indirect*, through final alignment accuracy. The paper does not answer three questions about the filter itself:

- *Do the captions describe the main content of the image?* BLIP produces unconditional captions: no prompt, no constraint on length or on what to describe. Nothing forces the caption to ignore background details. The paper says three captions are generated to reduce randomness, but `gene_mask.py` picks one of them with `random.choice` for the comparison, so the mask can differ between runs. Image noise is not removed; it becomes caption noise.
- *How does GPT ignore details?* The only mechanism is the question in the prompt (Table 12 of the paper): "do the two captions describe the same thing or topic, answer only Yes or No". Asking at the topic level is an implicit way to sidestep details, but no experiment measures how often GPT answers correctly. The binary answer also loses information: a half-relevant image and a completely wrong one both get an all-zero mask.
- *Is it better than a standard comparison?* The argument that "CLIP needs a hand-picked threshold" exists only in the text. The paper runs neither a CLIP filter nor the Masked-MMEA filter on the same data for comparison.

In addition, the mask is decided per entity *pair*. Table 8 of the paper reports 15,000 GPT calls for this step on DBP15K ZH-EN, exactly the number of gold pairs, while the seed set has only 4,500 pairs. This suggests masks were computed for the evaluation pairs as well, so the decision to keep or drop a test entity's image was made using its correct counterpart. If so, part of the test labels leaked into the input features. The procedure also cannot be applied to a new entity, since its counterpart is not yet known.

What we could verify. We could not run the filter: the authors did not release the mask file, and the roughly 50 GB of raw images are only on Baidu Cloud. However, our *unmasked* baseline reaches 0.732, above the 0.714 reference the paper

uses to measure the filter's contribution. Keeping the 0.801 ceiling, the filter's credit becomes about 6.9 points instead of 8.7. This compares two different implementations, so it does not refute the filter, but it shows that the 8.7 figure depends on the baseline, while the paper reports no run-to-run variation.

Experiments needed. (a) Hand-label a few hundred image pairs as relevant or not and measure the filter's precision and recall. (b) Run a CLIP filter (threshold chosen on a dev set) and a random filter that removes the same fraction of images, to tell whether the gain comes from *choosing the right images* or merely from *using fewer images*. (c) Compute masks on seed pairs only, or per entity rather than per pair, and measure again.

C. K3. "Near-optimal filtering ratio" and "only about 10 % of images"

The paper says. The filter reaches a near-optimal filtering ratio without manual tuning (Section 4.1, citing Table 4); among the contributions, LGEA reaches SOTA "using only about 10 % of the images compared with other methods".

Evidence. Table 4: LGEA 0.801, 0%mask 0.714, 50%mask 0.790, 80%mask 0.766 on FBDB15K.

Why it is not enough. The paper does not say what fraction of images the GPT filter actually keeps, so we cannot tell where LGEA sits on the ratio axis of Table 4, or what it is "near-optimal" relative to. Nor does it say how the 50% and 80% levels were produced (random selection or GPT scores). The label definition also contradicts itself: Section 5.4 defines the percentage as "the fraction of images considered valid (mask = 1)", but immediately calls 0%mask "no mask applied, all images kept". Under the first definition, 0% would mean dropping every image. Finally, the "about 10 %" figure appears in no table or experiment.

Experiment needed. Report the fraction of images the filter keeps on each dataset, and plot Hits@1 against the keep ratio using a single selection method, marking LGEA's point.

D. K4. Is "no entity names" guaranteed?

The paper says. In three places the paper states that the pipeline never uses surface forms, because names are removed from triple identifiers before reaching the LLM.

Why it is not enough. In DBP15K the entity name still exists *as an attribute value*: triples with the `name` predicate carry exactly the display name. Removing names from identifiers does not remove them from values, and the summarizing LLM copies the name into its description.

What we could verify. The attribute vector alone, with no learning, no structure and no image, reaches Hits@1 0.872 on ZH-EN, 0.845 on JA-EN and 0.853 on FR-EN (Table III), on par with LoginMEA, the strongest pre-LGEA method in the paper's own Table 1. We do not suggest this was intentional; the point is that "removing identifiers" does not by itself guarantee "no names". The FB datasets, whose attributes are mostly numeric, are a cleaner test, and that is also where LGEA loses to TMEA on FBDB15K.

E. K5. Does SOTA come from the two LLM modules?

Iterative learning adds 13 points but is not described. Removing iterative learning costs us 13.3 points Hits@1 on FBDB15K (paper: 11.7), more than images and relations combined in our measurements. The jump appears exactly at epoch 500, matching `il_start`. This mechanism is absent from Fig. 1 and Section 4 of the paper and mentioned only in one sentence of the setup, and the paper does not say whether MEAformer in Table 13 was run with the same iterative mode. The gap to earlier methods therefore cannot be credited entirely to the two LLM modules.

Two number sets for the same setting. Tables 2 and 13 report LGEA at 0.801/0.910/0.842 on FBDB15K with 20 % seed. The text of Appendix A.4 instead says LGEA reaches 0.771/0.894/0.816 and beats MEAformer by 0.193/0.082/0.155; adding these margins to MEAformer's numbers in Table 13 (0.578/0.812/0.661) gives exactly 0.771/0.894/0.816. The 3-point gap between the two sets is as large as the credit given to the image filter.

F. Where we agree with the paper

The importance ranking of the six components matches exactly between the paper and our reproduction, despite different hardware and attribute files (Table V). Graph structure remains the decisive signal, and the combination "LLM summary + LLM2Vec encoding" is clearly better than bag-of-attributes. The paper's main direction, cleaning the inputs before fusing modalities, is sound and supported by our reproduction. What is not established is the specific contribution of each LLM module, because the paper's experiments are indirect and change several factors at once.

VI. MAKAR: the grounded multimodal NER task

A. Problem statement

Given a tweet with sentence T and image I, GMNER [17] asks for the set of triples

$$F = \{(e_i, t_i, c_i)\}_{i=1}^{N}, \tag{5}$$

where e_i is a span of T, t_i is the entity type, and c_i is a bounding box (x_1, y_1, x_2, y_2) in I if the entity appears in the image, or None otherwise. FMNERG [18] is the same task with 51 fine-grained types (singer, athlete, park, …) instead of 4 coarse ones. For example, the sentence "The geometry of plants. Garfield Park Conservatory" with a photo of a greenhouse yields $\{(\text{Garfield Park Conservatory},\ \text{park},\ [x_1, y_1, x_2, y_2])\}$.

Three F1 metrics correspond to three levels of difficulty: MNER scores only span and type; EEG scores span and box (IoU > 0.5, or both None); MNERG scores the whole triple. Since MNERG requires every part to be correct at once, it is always the lowest of the three and is the main metric.

B. The two difficulties the paper targets

The first is *semantic ambiguity*: the same string can be an organization or a location depending on context ("Switzerland Co"), and the long-tailed distribution leaves many fine-grained types with very few training examples, so a sequence tagger

gets spans or types wrong. The second is that *entity names are too short for grounding*: unlike ordinary visual grounding, which has a descriptive phrase ("the man in the blue shirt on the left"), here there is only "Clint Dempsey", and the grounding model has to infer what that person looks like in the image.

C. Relation to prior work

H-Index [17] and TIGER [18] take candidate regions from an object detector and generate triples with a seq2seq model. RiVEG [19] marks the move to LLMs: an LLM rewrites entity names into descriptive phrases, which are then passed to a grounding model. MAKAR goes further in two directions: it uses a multimodal LLM as a knowledge source both *before* NER (auxiliary knowledge) and *after* it (entity correction with web search), and it replaces the grounding model with a multimodal LLM trained to reason step by step before giving coordinates.

VII. MAKAR: method

Fig. 2 redraws the three agents. In our reading, "multi-agent" here means three modules that call a multimodal LLM (MLLM) in different roles and run in sequence, not agents that negotiate with each other; the second agent does, however, branch on the output of the first.

A. KEA: enriching the input with the MLLM's internal knowledge

KEA does not replace the NER model; it changes its *input*. For each tweet, the MLLM is shown the three most similar samples (retrieved with BGE-VL from a 200-sample Query Set chosen by K-Means) and the definitions of the entity types, and is asked to write a paragraph of auxiliary knowledge: which entities may appear, of which type, and why. That paragraph is appended to the original sentence:

$$X = (s_1, \ldots, s_{N_1}, \langle \text{SEP} \rangle, aux_1, \ldots, aux_{N_2}), \qquad (6)$$

and X goes through XLM-RoBERTa [20] and a CRF [21] as in ordinary NER, with loss $-\log P(y^*|X)$. Only the labels of $s_{1..N_1}$ are scored; the aux part serves as context. In essence this is knowledge retrieval where the "knowledge base" is the MLLM's weights, and showing three similar samples is retrieval-augmented few-shot prompting.

B. ECA: correcting low-confidence entities with external knowledge

From the CRF, each predicted entity has a boundary probability p_b and a type probability p_t. ECA branches on a threshold:

$$E^{\text{Final}} = \begin{cases} \text{MLLM}_{\text{Correct}}(Res, Back, I, T) & \min(p_b, p_t) < \tau, \\ Res & \text{otherwise}, \end{cases} \qquad (7)$$

with $\tau = 0.5$. For a low-confidence entity, ECA searches the web with the original sentence, takes three pages, asks the MLLM to summarize them into $Back$, and then asks the MLLM to look at $Back$, the image, the sentence and the KEA output and rewrite the span and type. Note that ECA runs only on a small subset; its usefulness depends on whether the CRF "knows when it is unsure". Section VIII measures this.

Table VII: GRPO rewards and advantages ($G = 5$, $\alpha = 0.5$) for the entity "Nicki Minaj", gold box $[1, 16, 222, 245]$, sampled from the MAKAR-3B checkpoint. The fourth answer is well formatted but in the wrong region, so it is pushed down strongly; the other four are pushed up slightly. For groups where all five answers are correct, every $A_i = 0$.

i	Predicted box	r_{format}	r_{acc}	r_i	A_i
1	$[2, 15, 303, 238]$	1	1	1.0	$+0.5$
2	$[3, 16, 226, 237]$	1	1	1.0	$+0.5$
3	$[3, 15, 232, 237]$	1	1	1.0	$+0.5$
4	$[209, 14, 381, 237]$	1	0	0.5	-2.0
5	$[3, 15, 238, 237]$	1	1	1.0	$+0.5$

C. ERGA: grounding through step-by-step reasoning

ERGA takes each (e_i, t_i) and asks the MLLM a fixed question: "is e_i (a t_i) in the image? If so, give its coordinates; if not, return $[0, 0, 0, 0]$", with an instruction to ignore text in the image. The model must answer in the format $\langle$think$\rangle \ldots \langle$/think$\rangle\langle$answer$\rangle \ldots \langle$/answer$\rangle$. The think part is where the short entity name is expanded into a description: who, wearing what, standing where. To teach the MLLM to do this, the authors train in two stages in the spirit of DeepSeek-R1 [22]:

SFT cold start. The training data is split into easy and hard by letting Qwen2.5-VL-7B [23] predict boxes directly: samples with IoU > 0.5 are easy. Qwen2.5-VL-32B writes reasoning chains for both sets, which QwQ and DeepSeek-R1 then filter. 3,000 easy samples are used for SFT, teaching the model the answer format and the style of reasoning.

GRPO on hard samples. For each input the policy samples G answers $o_1..o_G$, each scored as

$$r = \alpha\, r_{\text{format}} + (1 - \alpha)\, r_{\text{acc}}, \qquad r_{\text{acc}} = \mathbb{1}[\text{IoU} > 0.5], \quad (8)$$

with $\alpha = 0.5$. GRPO [24] needs no value model: the advantage of each answer is its reward normalized within the group,

$$A_i = \frac{r_i - \text{mean}\{r_1..r_G\}}{\text{std}\{r_1..r_G\}}, \qquad (9)$$

and the objective is PPO-clip [25] on the probability ratio $\pi_{\text{new}}(o_i)/\pi_{\text{old}}(o_i)$ times A_i, minus $\beta\, \text{KL}(\pi_{\text{new}} \| \pi_{\text{SFT}})$ with $\beta = 0.01$ to stay close to the SFT model. Table VII illustrates (8)–(9) on a real sample we ran with the authors' checkpoint: there is a learning signal only when the group contains both correct and wrong answers; five correct answers give $A_i = 0$ for every i.

VIII. MAKAR: reproduction

A. Scope and forced deviations

The repository [26] provides the NER data with auxiliary knowledge attached, the NER training config, and the 3B ERGA checkpoint on ModelScope. Gold labels and boxes

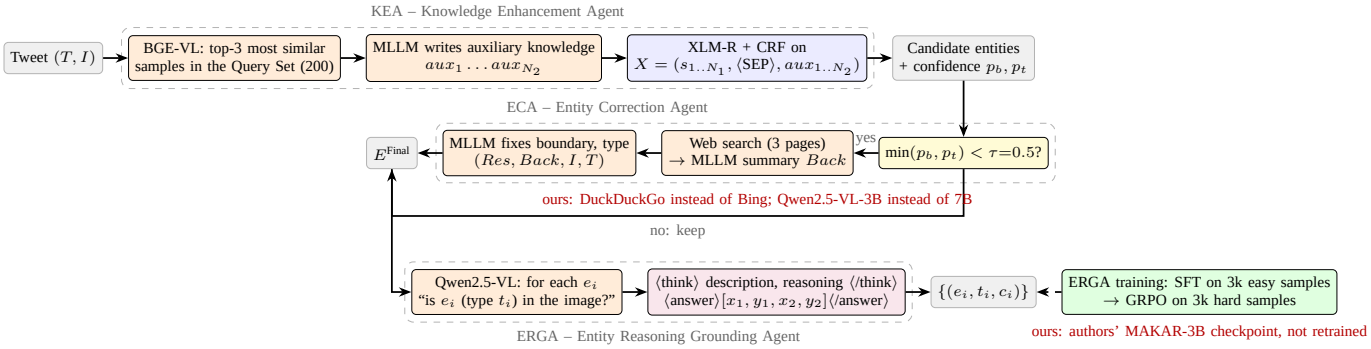


Figure 2: The three MAKAR agents and the data passed between them, as we read Fig. 3 of the original paper. KEA and ECA answer "which entity, which type"; ERGA answers "where in the image". Red notes mark where we had to deviate.

come from the original GMNER and FMNERG repositories; the Twitter-2015/2017 images (1 GB) come from Google Drive. What we actually reran: KEA training from scratch, all of ECA, and ERGA inference on the full test set. What we could not redo: SFT and GRPO for Qwen2.5-VL, because two T4 cards cannot train a 3B model with images; we use the official checkpoint `soliton110/MAKAR-3B`. Table VIII lists the differences. Each dataset ran in one Kaggle session (3.2 hours for FMNERG, about 3.8 hours for GMNER) with the same notebook MAKAR/notebooks/makar-reproduce-kaggle.ipynb in our repository [3], changing only `DATASET`; the GMNER kernel log is in MAKAR/logs/. W&B logs for FMNERG: NER training with KEA and without KEA; final results MAKAR, *w/o ECA*, *w/o KEA*; 300-sentence subset Raw 3B and MAKAR-3B. For GMNER: NER training with KEA and without KEA; final results MAKAR, *w/o ECA*, *w/o KEA*; subset Raw 3B and MAKAR-3B.

Each setting was run only once, so differences below about 2 F1 points in this section are not conclusive.

B. Table 2: data statistics

Before running any model, we recounted the data from the original labels (Table IX). All three columns match the paper exactly on both test sets, with no missing images. This check is cheap but rules out differences caused by using another version of the data.

C. FMNERG: full results

Table X is the main result. We *exceed* the paper on MNERG (63.75 vs. 60.54) and MNER (77.22 vs. 71.24), despite a smaller model and 2.5 times fewer NER epochs; EEG is 2.1 points below the 7B model but 1.1 points above the paper's 3B model in its Table 5. Exceeding the paper on MNER was unexpected and is explained in Section IX: our model is not better; the auxiliary knowledge for the test set is flawed.

The two ablation rows agree with the paper on *direction*: removing KEA hurts a lot (MNERG loses 7.8 points), removing ECA hurts little (0.2 points). On *magnitude*, however, the paper reports a 2-point MNERG loss without ECA and we measure only 0.2. Table XI shows why: the CRF is very confident, with 1,663 of 2,568 predicted entities above 0.99 confidence and only 91 entities, in 84 sentences, below $\tau = 0.5$. ECA changes 46 of those sentences and raises MNER from 76.93 to 77.22. Although the effect is small, ECA's underlying assumption holds: precision is 19.8 % in the low-confidence group versus 78.6 % in the high-confidence group, so the low-confidence group really is where corrections are needed.

For ERGA, the 3,151 grounding queries took 73 minutes on two cards, with 99.7 % of outputs in the correct think/answer format. Table XII compares the MAKAR-3B checkpoint

Table VIII: MAKAR reproduction setup and differences from the paper.

	Paper	Ours
Hardware	8×A800, bf16	Kaggle T4×2, fp16, transformers (vLLM does not install on Python 3.13)
NER (KEA)	20 epochs, extended train set of ≈28k sentences	8 epochs, exactly the 7,000 training sentences of the dataset
ECA search	Bing + ReaderLM-v2	DuckDuckGo titles + snippets, since the Bing Search API was retired
ECA MLLM	Qwen2.5-VL-7B	Qwen2.5-VL-3B-Instruct (7B does not fit on one card)
ERGA	SFT + GRPO, main results with 7B	authors' MAKAR-3B checkpoint, same prompt as their training data
Runs	mean of 3, no deviation reported	1 run

Table IX: Test-set statistics (paper / our recount from the raw data). The counts match exactly.

Test set	Entity	Grounded	Box
Twitter-GMNER	2 543 / 2 543	1 036 / 1 036	1 244 / 1 244
Twitter-FMNERG	2 543 / 2 543	1 046 / 1 046	1 254 / 1 254

Table X: Twitter-FMNERG, F1 (%): numbers from Tables 3 and 4 of the original paper (7B) versus ours (3B, ERGA not retrained). The paper does not report MNER or EEG for the two ablation rows.

	MNERG			MNER			EEG		
System	Paper	Ours	Δ	Paper	Ours	Δ	Paper	Ours	Δ
MAKAR (KEA+ECA+ERGA)	60.54	63.75	+3.2	71.24	77.22	+6.0	75.66	73.57	-2.1
w/o ECA	58.57	63.51	+4.9	–	76.93	–	–	73.57	–
w/o KEA	57.50	56.00	-1.5	–	67.59	–	–	71.94	–

Table XI: Marginal CRF confidence of entities predicted by KEA on the FMNERG test set, with precision per bin. The ECA threshold $\tau = 0.5$ catches only 91 entities.

Confidence bin	Entities	Precision (%)
$[0.0,\ 0.3)$	15	0.0
$[0.3,\ 0.5)$	76	23.7
$[0.5,\ 0.7)$	137	35.0
$[0.7,\ 0.9)$	220	50.0
$[0.9,\ 0.99)$	457	65.6
$[0.99,\ 1.0)$	1 663	89.6

Table XII: EEG of the 3B grounding model. The paper (Table 5) uses the full test set. We evaluate MAKAR-3B on the full test set to compare with the paper, and both models on the same random 300-sentence subset to compare them with each other; the Raw baseline was run on the subset only.

	Paper	Ours	
Grounding model	full test	full test	300 sent.
GMNER			
Qwen2.5-VL-3B (Raw)	22.45	–	22.95
MAKAR-3B (SFT + GRPO)	72.48	72.58	72.60
FMNERG			
Qwen2.5-VL-3B (Raw)	38.38	–	23.54
MAKAR-3B (SFT + GRPO)	72.43	73.57	74.98

with the untrained Qwen2.5-VL-3B-Instruct on the same 300 sentences: SFT + GRPO raises EEG from 23.5 to 75.0 on FMNERG and from 23.0 to 72.6 on GMNER. The base model mostly answers in the wrong format or returns a box covering the whole image. Our Raw numbers agree with the paper on GMNER (22.95 vs. 22.45) but not on FMNERG (23.54 vs. 38.38); we return to this in Section IX-C.

D. GMNER: full results

Table XIII gives the GMNER results. Here, where the auxiliary knowledge shows no sign of leakage, we are *below* the paper's 7B system on all three metrics, by 2.0 to 2.6 points. The comparison that isolates our deviations is EEG with the same 3B grounding model: 72.58 against the paper's 72.48 (Table 5 of the paper), a match to 0.1 points. The MNER gap (84.37 vs. 86.38) is consistent with our smaller NER training: 8 epochs on the 7,000 standard sentences instead of 20 epochs on the authors' extended file (Section IX-E).

The two ablation rows are where GMNER becomes informative. Our *w/o ECA* (69.24) and *w/o KEA* (67.87) match the paper's rows (69.23 and 67.63) to within 0.25 points, even though our full system is 2.6 points below theirs. In our run, ECA adds only 0.05 MNERG: the CRF flags just 10 entities, in 10 sentences, as below $\tau = 0.5$, and ECA changes 6 of those sentences (low-confidence precision 20.0 %, high-confidence 84.2 %). KEA adds 1.4 MNERG and 1.8 MNER. Section IX-B discusses what the match of the ablation rows implies.

The first GMNER attempt, before this run, stalled at 776 of 2,879 grounding queries because the authors' checkpoint ships with `use_cache=false`, left over from training; HuggingFace `generate` then recomputes the whole sequence, image included, for every new token, at about 80 seconds per query. With the cache re-enabled, grounding runs at about 1.4 seconds per query.

IX. MAKAR: analysis and critique

How to read this section. As with LGEA, we do not judge MAKAR by how high its numbers are but claim by claim: which experiment the authors give, whether that experiment isolates the contribution of the module being credited, and what we could verify ourselves. Table XIV summarizes five claims; each subsection M1–M5 follows the same order: *the paper says*, *evidence*, *why it is not enough*, *what we could verify*, *experiments needed*.

Table XIII: Twitter-GMNER, F1 (%): numbers from Tables 3 and 4 of the original paper (7B) versus ours (3B, ERGA not retrained, 7,000 training sentences). The paper reports EEG 72.48 for its own 3B model (Table 5).

	MNERG			MNER			EEG		
System	Paper	Ours	Δ	Paper	Ours	Δ	Paper	Ours	Δ
MAKAR (KEA+ECA+ERGA)	71.88	69.29	-2.6	86.38	84.37	-2.0	74.64	72.58	-2.1
w/o ECA	69.23	69.24	+0.0	–	84.32	–	–	72.53	–
w/o KEA	67.63	67.87	+0.2	–	82.54	–	–	71.63	–

Table XIV: The five main claims of MAKAR: the authors' evidence, what that evidence does not establish, and our verdict.

Claim	Evidence → gap	Verdict
M1. The MLLM's internal knowledge (KEA) resolves polysemy	*w/o KEA* drops 3 MNERG points → removing KEA removes both the knowledge and the retrieved examples; FMNERG test knowledge mentions gold labels (IX-A)	Not established on FMNERG
M2. Web search (ECA) fixes low-confidence entities	*w/o ECA* drops 2 points → $\tau = 0.5$ catches only 3.5 % of entities and is never tuned; we measure 0.2 (FMNERG) and 0.05 (GMNER) points (IX-B)	Right idea, small credit
M3. Step-by-step reasoning (CoT) enriches short entity names	Table 5 → no direct box SFT without CoT; unclear whether CoT was written with the gold box visible (IX-C)	Not isolated
M4. SFT on easy then GRPO on hard samples works best	Tables 5, 6 → we observe that GRPO on easy samples gives almost no gradient (IX-D)	Holds
M5. SOTA on GMNER and FMNERG	Table 3, "mean of 3 runs" → no standard deviation; the extended GMNER train set overlaps the test set (IX-E)	Partly true

A. M1. Does KEA really resolve semantic ambiguity?

The paper says. The MLLM serves as an implicit knowledge base: it writes auxiliary knowledge for each sentence, and RoBERTa-CRF reads the sentence together with that knowledge and can thus disambiguate polysemous entities (Section 3.2 of the paper).

Evidence. The *w/o KEA* row in Table 4 drops 4.3 MNERG points on GMNER and 3.0 on FMNERG; Fig. 7 shows that MNER changes with the MLLM used, Qwen2.5-VL-7B being best.

Why it is not enough. First, KEA contains *two* things: the knowledge written by the MLLM, and the three nearest labelled examples retrieved by BGE-VL from the 200-sample query set. Removing KEA removes both, so we cannot tell whether the credit belongs to "internal knowledge" or to labelled examples similar to the current sentence. Second, the paper gives no analysis on polysemous entities; "resolving ambiguity" is inferred only from overall F1. Third, and most serious: in the auxiliary-knowledge file the authors released for the FMNERG test set, 821 of 1,500 samples (55 %) contain sentences such as *"identified as a park based on the context provided by the ground truth entities"*. For instance, the first test sample, "The geometry of plants. Garfield Park Conservatory", has the gold label (Garfield Park, park), and its knowledge reads *"Garfield Park (type: park) … based on the context provided by the ground truth entities"*; these can be found by searching for the string `ground truth`. Most likely the MLLM was shown the gold labels when generating the knowledge. The FMNERG dev set and both GMNER dev and test sets show no such sign (0 %).

What we could verify. Three consequences are consistent with the leakage hypothesis: KEA's F1 on the FMNERG test set (76.93) is higher than on dev (70.85), which is unusual; our reproduction beats the paper on MNER and MNERG despite a smaller model; and the gap between KEA and *w/o KEA* grows to 9.6 MNER points on FMNERG, whereas on GMNER (no leakage) it is only 1.8 MNER and 1.4 MNERG points (Table XIII). So KEA does help, but by one to two points, not ten. EEG is less affected because the image region is predicted by ERGA, which does not read the auxiliary knowledge. We do not consider MNER and MNERG on FMNERG, the paper's included, to be clean results.

Experiments needed. (a) Regenerate the FMNERG test knowledge without gold labels in the prompt. (b) Split KEA: retrieved examples only, knowledge only, and knowledge replaced by Wikipedia text of the same length. (c) Report F1 separately on polysemous or rare entities.

B. M2. Does ECA fix wrong entities?

The paper says. For entities with $\min(p_b, p_t) < \tau$, ECA retrieves three web pages, summarizes them into background knowledge, and lets the MLLM correct span and type; high-confidence entities are kept as is (Eq. 7 of the paper).

Evidence. The *w/o ECA* row drops 2.0 MNERG points on FMNERG and 2.7 on GMNER (Table 4).

Why it is not enough. ECA only touches entities below the threshold, so its contribution is bounded by how many there are, which the paper does not report. With a well-trained CRF, marginal probabilities are heavily skewed toward 1 (Table XI): on FMNERG only 91 of 2,568 entities (3.5 %) fall below 0.5. A 2-point F1 drop without ECA would require ECA to correctly fix most of these few entities, which is unlikely. The threshold $\tau = 0.5$ is fixed and never tuned. The search engine is not named either, so web results change over time and cannot be reproduced exactly.

The GMNER numbers make this concrete. The test set has 2,543 gold entities, so one F1 point corresponds to roughly 25 entities, and the paper's 2.65-point drop without ECA would need about 65 entities fixed by ECA. In our run, the CRF flags

only 10. Moreover, our *w/o ECA* (69.24) and *w/o KEA* (67.87) rows match the paper's (69.23, 67.63) almost exactly, while our full system is 2.6 points below the paper's 7B system. In other words, our ablation rows already sit where the paper's do, and the paper's full row is the one that differs. The paper does not say which grounding model its ablation rows used; if they used the 3B model while the full row used the 7B one, most of the "ECA" and "KEA" gaps in its Table 4 would be the 7B–3B difference, which the paper's own Table 5 puts at 2.16 EEG points on GMNER. We cannot confirm this, but it is the simplest explanation consistent with all the numbers we have.

What we could verify. With DuckDuckGo search and Qwen2.5-VL-3B instead of 7B, ECA changes 46 sentences and raises MNERG from 63.51 to 63.75 (0.2 points); on GMNER it changes 6 sentences and raises MNERG by only 0.05 points (69.24 to 69.29). Table XI also shows that confidence is genuinely informative: the group below the threshold is only 19.8 % correct, the group above it 78.6 %. The idea is right, but the current threshold leaves most errors untouched (the 0.5–0.9 band holds 357 entities that are only 35–50 % correct).

Experiments needed. (a) State which grounding model each ablation row uses, and rerun the ablations with the full system's model. (b) Sweep τ from 0.5 to 0.95 and plot F1 against the number of search calls. (c) Report how many entities ECA touches, fixes and breaks. (d) Compare with an MLLM correction step without search, to tell whether the credit belongs to the web or to the MLLM.

C. M3. Is step-by-step reasoning really why grounding improves?

The paper says. Entity names are short and carry little semantics; having the model reason step by step (describe the entity from text and image, then give coordinates) "enriches the semantics" and grounds more accurately (Section 3.4).

Evidence. Table 5: Qwen2.5-VL-3B goes from 22.45 EEG (raw) to 69.88 (SFT) and 72.48 (SFT then GRPO) on GMNER; the trained 3B model beats the untrained 32B model (68.91).

Why it is not enough. Every SFT row in Table 5 trains on CoT data. No row trains directly on "sentence + entity name $\rightarrow$ box" without reasoning, so the jump from 22 to 70 points may come mostly from learning the coordinate format and the box distribution of the Twitter data, not from reasoning. Furthermore, the CoT is written by Qwen2.5-VL-32B and filtered by QwQ and DeepSeek-R1, yet the 32B model only reaches 68.91 EEG when grounding on its own. The paper does not say whether the model was given the gold box when writing CoT; if it was, the CoT is an explanation written backwards from the answer rather than real reasoning, which should be stated. Finally, comparing against the untrained 32B model is not a fair comparison, since that model has never seen the Twitter data.

What we could verify. On 300 FMNERG sentences, the base 3B model with the training prompt reaches 23.54 EEG while the MAKAR-3B checkpoint reaches 74.98 (Table XII), confirming that the big jump is real. Without the resources to retrain, we cannot isolate the CoT's share. The paper's "Raw" row is also internally inconsistent. GMNER and FMNERG share the same images and boxes and differ only in type granularity, so EEG of a fixed model should be nearly equal on both: the paper's own 7B Raw (60.09 vs. 60.05) and MAKAR-3B (72.48 vs. 72.43) rows confirm this. Yet its 3B Raw row reads 22.45 on GMNER and 38.38 on FMNERG. We measure 22.95 and 23.54, matching the paper's GMNER value, which suggests the FMNERG entry is an error or used a different prompt that the paper does not describe.

Experiments needed. (a) SFT with the same number of samples but with box answers only, no CoT. (b) Fine-tune the 32B model the same way for a fair comparison. (c) State the CoT generation prompt and whether the gold box was provided.

D. M4. SFT on easy samples, then GRPO on hard samples

The paper says. SFT on easy samples teaches the reasoning format; GRPO on hard samples helps the model handle difficult cases. Easy samples are those that the base Qwen2.5-VL-7B grounds correctly (IoU > 0.5).

Evidence. Table 6: the Easy/Hard configuration reaches 74.64 EEG on GMNER, above the other three (67.82–70.47). Table 5: GRPO without SFT is far worse (45.18 for 3B).

Assessment. We find this claim holds, because it matches how GRPO works. The advantage in GRPO is the reward normalized within the group (Eq. 9); if all G answers are equally right or equally wrong, the advantage is 0 and there is no gradient. When we sampled five answers for six entities with the authors' checkpoint, most groups had all five answers with the same reward; only the "Nicki Minaj" group produced a signal (Table VII). On easy samples GRPO has almost nothing to learn, exactly as Table 6 shows.

What is still missing. GRPO adds 2.6 points (3B) and 3.0 points (7B) over SFT alone on GMNER; the paper says these are means of three runs but gives no standard deviation, so we do not know how much larger this gain is than run-to-run variation. The easy/hard split is defined with the 7B model but reused for the 3B model, for which many "easy" samples may already be hard. The paper also does not report the fraction of groups with non-zero advantage, the most direct number for checking this argument.

E. M5. Does MAKAR reach SOTA?

The paper says. MAKAR beats previous methods by 3.36 points on GMNER and 8.06 points on FMNERG (Table 3), each number being a mean of three runs.

Why it is not enough. First, no table gives a standard deviation, so we cannot tell whether small gaps (for example 0.44 MNER points on FMNERG) are meaningful. Second, the GMNER training file in the authors' repository has 28,070 lines but only 9,335 distinct sentences: the 7,000 GMNER sentences are mixed with Twitter-2015/2017 and duplicated across several knowledge versions, including 43 sentences that

also appear in the test set and `MISC` labels that are not part of the GMNER label set. The paper does not mention this extension. Third, on FMNERG, MNER and MNERG are affected by the label leakage in M1. Fourth, the resources are not comparable: MAKAR uses a 7B MLLM for KEA and ECA, web search, and CoT data produced by 32B, QwQ and DeepSeek-R1 models, none of which the baselines have. This is not a flaw in itself, but the margin should be read as "a larger system does better", not only "a better design". Finally, ERGA needs one reasoning generation for *each* entity (we measure 1.4 seconds per entity on a T4 with the 3B model); the paper acknowledges it is slow but gives no number.

What we could verify. EEG, which does not depend on auxiliary knowledge, reproduces closely with the same 3B grounding model: 72.58 versus 72.48 on GMNER and 73.57 versus 72.43 on FMNERG. Our notebook uses only the 7,000 GMNER training sentences and reaches 84.37 MNER versus the paper's 86.38; this gap is partly due to less data and partly to the absence of the 43 overlapping sentences. In addition, the released checkpoint ships with `use_cache=false`, which makes inference far slower unless the user fixes it.

Experiments needed. Report standard deviations; train on the standard GMNER training set, or state the extension and remove test overlaps; and add inference time per sentence to Table 3.

F. Where we agree with the paper

Two main conclusions hold. Auxiliary knowledge from an MLLM does help NER, but by about 1.8 MNER points as measured on GMNER, where there is no leakage, not by the nearly 10 points seen on FMNERG. Two-stage reasoning training turns a 3B model from nearly useless for grounding into one on par with the paper's 7B model, although the CoT's own share has not been isolated. What should be redone is the FMNERG table with auxiliary knowledge generated cleanly, without gold labels, and the ablation table with the same grounding model as the full system.

X. Conclusion

We reproduced two EMNLP 2025 papers with a small fraction of the authors' compute. For LGEA, we ran Table 1 on three splits and the seven ablation settings of Table 3, reaching Hits@1 within 1–2.5 points of the paper with the importance ranking matching 6 of 6, but we had to rebuild all attribute data and could not run the image filter because the authors did not release the masks. For MAKAR, we fully ran both datasets. Grounding with the same 3B model matches the paper to about 0.1–1.1 EEG points. On FMNERG we obtained an MNERG above the paper and found signs that the cause lies in the test-set auxiliary knowledge, which contains sentences referring to the gold labels. On GMNER, where there is no such sign, we are 2.6 MNERG points below the 7B system, yet our ablation rows match the paper's almost exactly, which questions how much of the paper's ablation gaps belong to ECA and KEA.

Three lessons are common to both papers. First, the improvement credited to a new module is often sensitive to mechanisms that are barely mentioned: iterative learning in LGEA, the extended training set in MAKAR. Second, "not using information X" must be checked on the data, not only in the code: entity names sit in attribute values, gold labels sit in the auxiliary-knowledge text. Third, a paper's code release should be enough to run from scratch; both repositories lack preprocessing outputs or have packaging bugs, and most of our reproduction time went into that rather than into the models.

Remaining work is to run FBYG15K for LGEA, and for MAKAR to regenerate the FMNERG auxiliary knowledge without gold labels to obtain a clean FMNERG table.

References

[1] C. Lu, C. Li, J. Cheng, Y. Ji, G. Chen, and F. Zhang, "Breaking the noise barrier: LLM-guided semantic filtering and enhancement for multi-modal entity alignment," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2025, pp. 33 153–33 167.

[2] X. Lin, Y. Zhang, Y. Xu, K. Huang, H. Mu, Y. Wang, G. Gou, L. Qian, L. Peng, W. Liu, J. Luan, and H. Xu, "MAKAR: a multi-agent framework based knowledge-augmented reasoning for grounded multimodal named entity recognition," in *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing (EMNLP)*. Association for Computational Linguistics, 2025, pp. 6110–6130.

[3] N. H. Châu and M. T. Hoàng, "Reproduction code for LGEA and MAKAR," https://github.com/MinhThang03/graphML, 2026.

[4] Z. Sun, W. Hu, and C. Li, "Cross-lingual entity alignment via joint attribute-preserving embedding," in *The Semantic Web – ISWC 2017*. Springer, 2017, pp. 628–644.

[5] Y. Liu, H. Li, A. Garcia-Duran, M. Niepert, D. Onoro-Rubio, and D. S. Rosenblum, "MMKG: Multi-modal knowledge graphs," in *The Semantic Web (ESWC)*. Springer, 2019, pp. 459–474.

[6] Z. Lin, Z. Zhang, M. Wang, Y. Shi, X. Wu, and Y. Zheng, "Multi-modal contrastive representation learning for entity alignment," in *Proceedings of the 29th International Conference on Computational Linguistics (COLING)*, 2022, pp. 2572–2584.

[7] Z. Chen, J. Chen, W. Zhang, L. Guo, Y. Fang, Y. Huang, Y. Zhang, Y. Geng, J. Z. Pan, W. Song, and H. Chen, "MEAformer: Multi-modal entity alignment transformer for meta modality hybrid," in *Proceedings of the 31st ACM International Conference on Multimedia*, 2023, pp. 3317–3327.

[8] Y. Shi, M. Zhang, M. Wang, Z. Lin, X. Wu, and Y. Zheng, "Probing the impacts of visual context in multimodal entity alignment," in *Web and Big Data (APWeb-WAIM)*. Springer, 2022.

[9] L. Chen, Y. Sun, S. Zhang, Y. Ye, W. Wu, and H. Xiong, "Tackling uncertain correspondences for multi-modal entity alignment," in *Advances in Neural Information Processing Systems (NeurIPS)*, 2024.

[10] J. Li, D. Li, C. Xiong, and S. Hoi, "BLIP: Bootstrapping language-image pre-training for unified vision-language understanding and generation," in *Proceedings of the 39th International Conference on Machine Learning (ICML)*, 2022, pp. 12 888–12 900.

[11] A. Grattafiori *et al.*, "The Llama 3 herd of models," arXiv:2407.21783, 2024.

[12] P. BehnamGhader, V. Adlakha, M. Mosbach, D. Bahdanau, N. Chapados, and S. Reddy, "LLM2Vec: Large language models are secretly powerful text encoders," in *First Conference on Language Modeling (COLM)*, 2024.

[13] P. Veličković, G. Cucurull, A. Casanova, A. Romero, P. Liò, and Y. Bengio, "Graph attention networks," in *International Conference on Learning Representations (ICLR)*, 2018.

[14] C. Lu, "LGEA-framework: official code," https://github.com/alusang/LGEA-framework, 2025, accessed 2026-10-29.

[15] A. Conneau, G. Lample, M. Ranzato, L. Denoyer, and H. Jégou, "Word translation without parallel data," in *International Conference on Learning Representations (ICLR)*, 2018.

[16] T. Su, J. Sheng, S. Wang, X. Zhang, H. Xu, and T. Liu, "LoginMEA: Local-to-global interaction network for multi-modal entity alignment," in *Proceedings of the 27th European Conference on Artificial Intelligence (ECAI)*, 2024.
[17] J. Yu, Z. Li, J. Wang, and R. Xia, "Grounded multimodal named entity recognition on social media," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023, pp. 9141–9154.
[18] J. Wang, Z. Li, J. Yu, L. Yang, and R. Xia, "Fine-grained multimodal named entity recognition and grounding with a generative framework," in *Proceedings of the 31st ACM International Conference on Multimedia*, 2023, pp. 3934–3943.
[19] J. Li, H. Li, D. Sun, J. Wang, W. Zhang, Z. Wang, and G. Pan, "LLMs as bridges: Reformulating grounded multimodal named entity recognition," in *Findings of the Association for Computational Linguistics: ACL 2024*, 2024.
[20] A. Conneau, K. Khandelwal, N. Goyal, V. Chaudhary, G. Wenzek, F. Guzmán, E. Grave, M. Ott, L. Zettlemoyer, and V. Stoyanov, "Unsupervised cross-lingual representation learning at scale," in *Proceedings of the 58th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2020, pp. 8440–8451.
[21] J. Lafferty, A. McCallum, and F. C. N. Pereira, "Conditional random fields: Probabilistic models for segmenting and labeling sequence data," in *Proceedings of the 18th International Conference on Machine Learning (ICML)*, 2001, pp. 282–289.
[22] DeepSeek-AI, "DeepSeek-R1: Incentivizing reasoning capability in LLMs via reinforcement learning," arXiv:2501.12948, 2025.
[23] S. Bai *et al.*, "Qwen2.5-VL technical report," arXiv:2502.13923, 2025.
[24] Z. Shao, P. Wang, Q. Zhu, R. Xu, J. Song, X. Bi, H. Zhang, M. Zhang, Y. K. Li, Y. Wu, and D. Guo, "DeepSeekMath: Pushing the limits of mathematical reasoning in open language models," arXiv:2402.03300, 2024.
[25] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," arXiv:1707.06347, 2017.
[26] X. Lin, "MAKAR: official code," https://github.com/Nikol-coder/MAKAR, 2025, accessed 2026-10-29.

Appendix A
Six patches needed to run the LGEA code

1) `ModuleNotFoundError` for `transformers.modeling_layers`. McGill's LLM2Vec repository rewrote `modeling_llama_encoder.py` in late 2025 and requires `transformers`$\geq$4.56, while `llm2vec` pins $\leq$4.44; the old version in turn needs `LlamaFlashAttention2`, removed in 4.48. Fix: pin the repository's `code_revision` to a compatible version.
2) The layer-split model cannot be pickled when `llm2vec` spawns one process per GPU. Fix: a separate `encode_sharded()` function, with ordering invariance checked on 4 cases.
3) The second PEFT adapter loaded after `merge_and_unload()` sits on the wrong device. Fix: realign the 224 adapter layers with PEFT's internal helper.
4) `accelerate` splits a `ModifiedLlamaDecoderLayer` across the two cards because of class-name matching. Fix: an explicit `device_map`, balanced at 7.59 and 7.42 GB.
5) OOM of 5.66 GiB in `layers.py`: the gradient of the sparse matrix product builds a dense $N \times N$ matrix. Fix: rewrite it per edge, $O(N^2) \rightarrow O(E \cdot d)$, using 0.08 GiB; equivalence verified in plain Python before use.
6) The log parser looks for `hits@1` but the actual log prints `acc of top [1, 10, 50]`. Fix: rewrite the regular expression.

Appendix B
LGEA attribute-summarization prompt (Table 11 of the paper)

The prompt asks LLaMA-3 to write an English description of an entity from its (attribute, value) pairs, with one worked example and four main constraints:

- at most 100 words;
- summarize only the given information;
- add no details beyond describing the entity;
- do not state the entity's name.

The full text is in Table 11 of the original paper [1].

Appendix C
Work split and links

Table XV summarizes how work was split within the team.

Table XV: Work split within the team.

Member	Tasks
Hoàng Minh Thắng (25C11020)	• Ran the LGEA and MAKAR experiments on Kaggle • Built the data, patched the original code • Logged runs to W&B, compiled the numbers
Châu Ngọc Huy (25C11008)	• Studied the methods of both papers • Analysis and critique of the papers • Made the presentation slides
Both	• Wrote the report

Reproduction code and rerun instructions: https://github.com/MinhThang03/graphML [3]. Public W&B experiment logs: LGEA at https://forge.coreweave.com/wandb/chaungochuy0709-university-of-science-vietnam-national-/graphml-lgea, MAKAR at https://forge.coreweave.com/wandb/chaungochuy0709-university-of-science-vietnam-national-/graphml-makar. The runs were imported from the Kaggle notebook outputs after the fact (models were not rerun); each run's notes give the source notebook and original run date.